Fermi: A Decentralized, Low-Latency, Verifiably Fair Ordering and Settlement Protocol for Orderbook DEXs

Fermi Labs

May 16, 2025

Abstract

Abstract: We present *Fermi*, a decentralized exchange protocol that implements a high-performance Central Limit Order Book (CLOB) with verifiable fairness in transaction ordering and settlement. Fermi is designed to support on the order of 10^5 transactions per second (TPS) in order placement and $\sim 10^4$ TPS in trade settlement, matching centralized exchange performance while preserving decentralization and trustlessness. The architecture combines geographically distributed ingress sequencer nodes, a global sequencing and matching layer (transitioning from a temporarily trusted sequencer to a provably correct execution inside a zkVM), and on-chain settlement on Solana. We detail how Fermi enshrines *price-time priority* and first-in-first-out (FIFO) fairness in its core design: orders are timestamped at the edges to prevent manipulation, and a forthcoming verifiable delay function (VDF) "braided timeline" will cryptographically enforce chronological integrity. Each order receives a signed receipt as proof of submission, enabling participants to later prove inclusion or flag omission. A global sequencer produces an ordered batch of matches, which will ultimately be accompanied by succinct proofs attesting that global ordering and matching followed the fairness rules. Trades are settled on Solana via smart contracts that verify user-signed order intents (ed25519 signatures) before transferring assets, ensuring non-custodial execution. We discuss alternative approaches and trade-offs in achieving low-latency fair ordering, and we provide a detailed description of Fermi's finalized system architecture, sequencing logic, fairness model, and settlement pipeline.

1 Introduction

Decentralized exchanges (DEXs) have become a cornerstone of the cryptocurrency ecosystem, enabling trustless trading without centralized custodians. However, most popular DEX designs (such as automated market makers) sacrifice the performance and features of traditional centralized exchanges (CEXs), particularly the high throughput and fine-grained control of a central limit order book (CLOB). On the other hand, attempts to implement orderbook-based DEXs on blockchain platforms face challenges in both *performance* and *fairness*. Throughput on most layer-1 blockchains is limited, making it difficult to support tens of thousands of orders per second required by active markets. Additionally, without special precautions, transaction ordering on a public blockchain is typically controlled by miners/validators, which opens the door to front-running and other manipulations of order sequence for profit. Indeed, miners or block producers can extract significant *Maximal Extractable Value (MEV)* by arbitrarily reordering, inserting, or censoring transactions in a block, often to the detriment of ordinary users who may have their trades front-run or sandwich-attacked. These phenomena have been extensively documented in prior work (e.g., *Flash Boys 2.0*), which highlighted how decentralized exchanges on Ethereum

suffered from pervasive front-running, leading to hundreds of millions of dollars in unfair gains and even threatening consensus-layer security.

Achieving a fair ordering of transactions in a decentralized setting is a non-trivial problem. Traditional exchanges solve ordering by operating a fully centralized order matching engine that simply sequences orders in the exact arrival order (enforcing a strict price-time priority). In that model, multiple gateway servers accept client orders and forward them to a central accumulator, which merges them into a single ordered stream according to timestamp (FIFO) and price priority. This centralized approach yields low latency and a clear fairness criterion (first-come, first-served among those at the best price). However, replicating this in a decentralized environment without trusting a single party is challenging. Naively putting an orderbook on-chain (as done by early protocols like Serum on Solana or certain Ethereum rollups) can preserve non-custodial execution, but miners/validators still control inclusion and ordering of on-chain transactions. This can lead to unfair outcomes unless additional mechanisms are introduced, and it also forces every order to pay on-chain fees and incur blockchain latency.

Researchers and practitioners have explored several approaches to enforce fair ordering in blockchain systems. One approach is to integrate fairness constraints into the consensus protocol itself, using techniques from distributed systems. For example, *Aequitas* (Kelkar et al.) and related academic works propose new consensus algorithms that achieve ordering fairness even in asynchronous networks, though often with high communication complexity (e.g. $O(n^3)$ in some cases) or stronger synchrony assumptions. Other works decouple the timestamping of transactions from the actual consensus ordering. The Decentralized Clock Network (DCN) by Constantinides et al. (2023) is one such proposal, in which a committee of nodes assigns each transaction a trusted timestamp via a Byzantine fault tolerant protocol, and then the blockchain orders transactions by these timestamps. By separating timing from final ordering, this approach can prevent blatant front-running (if tx_A is sufficiently earlier than tx_B, tx_B cannot be placed before tx_A in any block). Another class of solutions employs *frequent batch auctions* or encrypted transaction flow to level the playing field, processing all transactions in a short time window as a batch (thus prioritizing price over tiny differences in time). While batch auctions (such as those inspired by Budish's frequent call auctions) can mitigate fast arbitrageurs' advantage, they introduce artificial delay and may not satisfy traders who demand continuous price-time priority matching.

A recent promising direction is the use of *Trusted Execution Environments (TEEs)* and cryptography to enforce ordering policies off-chain, combined with on-chain verification. For example, Chainlink's *Fair Sequencing Service* and the IC3 *PROF* (Protected Order Flow) concept both involve transactions being sent to a trusted enclave which sequences them according to a fair policy and outputs an ordered bundle that validators must accept without reordering. A TEE (such as Intel SGX) can ensure that even if the sequencing service is run by a single entity, it cannot deviate from the prescribed fair ordering policy or reveal transaction details prematurely, assuming the hardware is secure. However, pure TEE-based solutions place trust in hardware and the TEE operator's honesty (or in remote attestation for correctness). Moreover, they often still rely on an honest majority of validators to ultimately enforce inclusion of the sequence (as in PROF's case where a block builder must commit to including the enclave-produced bundle).

Our Contribution: In this paper, we introduce **Fermi**, a decentralized orderbook exchange protocol that reconciles high throughput, low latency, and fairness through a novel combination of system design elements. Fermi's architecture explicitly enshrines *price-time priority* fairness while leveraging cryptographic proofs to eventually eliminate any single trusted component. The system consists of:

- **Local Ingress Sequencers:** A globally distributed network of ingress nodes that accept user

orders. Each ingress node obtains verifiable timestamps from time-synchronized third-party *Roughtime* servers. This yields low-latency order ingestion close to users (reducing network propagation delays) while providing a cryptographically certified timestamp to mitigate local tampering.

- **Signed Order Receipts:** Upon receiving an order, an ingress node returns a signed, timestamped receipt to the client. The timestamp is anchored by the Roughtime service, and the signed receipt serves as cryptographic evidence of when the order was accepted. Clients can use these receipts to prove inclusion (or unjust omission) of their orders in the global sequence, holding sequencers accountable.

- **VDF-Braided Timelines (Future):** In upcoming versions, each ingress node will embed its order stream into a verifiable delay function (VDF) chain. This creates a tamper-evident timeline: even a malicious node operator cannot arbitrarily adjust or backdate timestamps without breaking the VDF linkage. The "braiding" of VDF outputs from multiple ingress nodes—each anchored by independently obtained Roughtime proofs—may be used to derive a unified global time reference that is manipulation-resistant.

- **Global Sequencer and Matching Engine:** Fermi currently utilizes a logically centralized global sequencer that aggregates orders from all ingress nodes, enforces FIFO ordering and matching across the unified order book, and produces matched trades. In the current implementation this sequencer is a trusted component for liveness and correctness. However, the design is built to transition this role into a *zk-proven* service: the sequencer will run inside a Zero-Knowledge virtual machine (ZK-VM) which generates succinct proofs (specifically Groth16 SNARKs) that the ordering and matching computations were executed correctly according to the rules (no order was missed or reordered unfairly, and matches obey price-time priority).

- **On-Chain Settlement on Solana:** Trade results output by the global sequencer are settled on the Solana blockchain for finality and custody of assets. Each user's order is an off-chain intent accompanied by an Ed25519 signature (using the user's Solana public key). The on-chain Solana program verifies these signatures against the order details before moving any funds, meaning that no trade can occur without the user's explicit signed authorization. Consequently, Fermi never takes custody of user funds — assets remain under users' control in smart contracts, with the protocol only executing transfers that users have pre-signed and approved.

In the remainder of this paper, we delve into the rationale behind these design choices and explain in detail how the system works. In Section 2, we review related approaches to the problem of fair ordering and highlight the trade-offs of different designs (hardware-based vs cryptographic, on-chain vs off-chain matching, etc.). Section 3 presents the overall architecture of Fermi, describing the components and their interactions. We then focus on the **sequencing logic and fairness model** in Section 4: how local timestamps are generated and used, how the global order is constructed and proven fair, and how the system prevents malicious behaviors like front-running, order censorship, or timestamp manipulation. Section 3.4 details the settlement pipeline on Solana, including how orders are represented as signed intents, how trades are executed on-chain with signature verification to ensure non-custodial execution, and the performance considerations of Solana as a settlement layer. In Section **??**, we outline the path toward progressively decentralizing and hardening the protocol's security: eliminating the need to trust any single sequencer via zk-proofs and/or fraud proofs, introducing a network of sequencers that stake tokens and are subject to slashing for misbehavior,

and integrating economic security measures to incentivize honest participation. We emphasize how each evolutionary step maintains or improves the core properties of *performance*, *fairness*, and *verifiability*. Finally, Section 7 concludes with a summary of how Fermi advances the state-of-the-art in DEX design and a discussion of future work.

2 Background and Related Work

Achieving high-performance and fair ordering in a decentralized exchange has been the subject of much research and engineering effort. We briefly review the landscape of existing approaches, to frame how Fermi's design compares and why certain choices were made.

2.1 On-Chain Order Books vs. Off-Chain Matching

One straightforward way to build a DEX is to put the entire order book and matching process on-chain, using smart contracts to manage order placement, cancellation, and trade execution. This is the approach taken by Serum/OpenBook on Solana and some Ethereum-based DEXs: users submit transactions to the blockchain to place or cancel orders, and the matching of buy/sell orders occurs as part of on-chain transaction processing. The benefit of this model is its simplicity and trustlessness: the chain's consensus process itself determines the order of transactions (and hence of orders/trades), and settlement is atomic on-chain. However, performance is limited by the underlying blockchain's throughput and latency. For instance, even though Solana is a high-performance chain (capable of thousands of TPS in ideal conditions), an on-chain orderbook like Serum cannot easily reach the 100k TPS scale required for a truly high-frequency trading environment. Users also pay network fees for each order action, which can be prohibitive for market makers updating quotes frequently. Moreover, on-chain ordering does not inherently solve fairness – miners/validators can still reorder transactions in the mempool. Solana's architecture, with its leader schedule and *Proof of History* clock, mitigates this to some extent by timestamping transactions as they enter the leader's pipeline, but it does not eliminate the possibility of a malicious leader reordering or delaying certain user transactions.

Off-chain matching with on-chain settlement is an alternative adopted by protocols like 0x and many centralized matching / decentralized custody hybrids. In 0x protocol, for example, orders are created and signed by users off-chain and collected by relayers; when a matching counterparty is found, the relayer submits the pair of orders to an on-chain smart contract to settle the trade. This approach offloads the heavy work of order management from the blockchain, allowing much higher throughput (since off-chain communications can be high-frequency and don't incur gas costs) and lower latency (trades can be matched instantly by a server, without waiting for block confirmation). The on-chain component only sees the final matched trade for settlement. The obvious drawback is that a naive implementation introduces a trusted party (the off-chain matcher) who could reorder trades, ignore some users' orders, or even match at unfair prices. 0x attempts to remain trustless by allowing anyone to act as a relayer and by requiring user signatures on the exact trade execution, but in practice, if a relayer is used, users must rely on that relayer to behave fairly (or there must be a system of competing relayers).

Fermi falls into the off-chain matching category, but with strong measures to ensure the matcher cannot behave dishonestly without detection and punishment. Unlike a typical centralized order matcher, Fermi's sequencer will produce cryptographic proofs of correct behavior, and users have tools (like receipts and eventually on-chain proofs) to enforce fairness.

2.2 Fair Ordering Mechanisms

Ensuring fair ordering (typically defined as "first-come, first-served" or more formally, if event A occurs before event B in real time by some margin, then A should be ordered before B in execution) has been addressed via multiple mechanisms:

Consensus Protocol Modifications: As noted, protocols like Aequitas, Themis, and others modify BFT consensus to achieve order fairness. They often require committees to collect and compare timestamps or ordering preferences. While effective in theory, these protocols can introduce significant complexity or latency overhead and have not yet been adopted in mainstream permissionless blockchains. There is also the challenge of reconciling fairness with liveness in asynchronous networks.

Decoupled Timestamping: The approach of assigning trusted timestamps to transactions prior to ordering (as in the DCN) is attractive for its ability to lower consensus complexity. If one can trust the timestamp on each transaction, then ordering by timestamp yields a fair sequence. The trust can come from a decentralized timestamping service (assuming $>2/3$ are honest). Fermi's design uses a form of decoupled timestamping at the "edge" (ingress nodes) but relies on TEEs to make those timestamps credible, rather than an extensive cross-network protocol. Effectively, each ingress node is a trusted timestamping service for the transactions it sees. Combining these into a global order then requires trusting the collection of ingress nodes to some degree (or verifying them after the fact via proofs).

Encrypted Transactions and Fair Sequencing Services: Another strategy is to prevent sequencers from seeing transaction content until an order is fixed. This can be done via threshold encryption (users encrypt their transactions, which are only decrypted after an ordering is chosen) or commit-reveal schemes. Such techniques are used in some MEV mitigation strategies (e.g., threshold encryption of Ethereum mempool transactions, and the Chainlink FSS concept). These approaches can enforce a kind of fairness by hiding actionable info, but they often incur delays (one must wait for a batch to fill or a time window to close before decrypting) and may not satisfy the strict price-time priority rule, since they tend toward batch processing.

Chainlink's FSS/PROF, which we touched on, effectively combines encryption and TEE: users send transactions to a TEE which keeps them private and outputs an ordered list committed to by block producers. This requires some integration with the L1 (e.g., Ethereum block builders accepting the enclave's output). Fermi's environment is a bit different (it runs its own sequencer rather than piggybacking on an L1 block production process), but the principle of using TEEs for fair sequencing is similar.

Verifiable Delay Functions (VDFs): VDFs are cryptographic constructs that require a predetermined amount of sequential computation to produce a result, and whose result can be efficiently verified. VDFs (like iterated squaring or Solana's SHA256-based PoH) can serve as a source of time that is intrinsic to the system. Solana's Proof of History can be viewed as a sort of VDF that orders events by the output sequence of a hash chain. By continuously hashing and embedding events in the hash state, a Solana leader effectively timestamps transactions in a way that others can later verify (they can verify an event was at position X in the sequence by recomputing the hashes). However, in Solana, only the leader runs the PoH at any given time, so one must trust that leader not to manipulate input ordering (leaders rotate quickly, reducing risk, but a malicious leader in

its slot could still choose ordering arbitrarily — PoH just proves a timeline of *some* ordering, not that it was first-come-first-served).

Fermi's planned use of VDFs is to strengthen the trust in ingress nodes' timelines. By having each ingress node continuously compute a VDF and mix incoming order IDs into the VDF state (similar to how a Solana leader mixes transactions into its PoH hash), we create a verifiable log of what order events occurred and in what order. If an ingress node tried to back-date an order or insert a late order with an earlier timestamp, the VDF output would not match the expected result that others (who saw the sequence of VDF outputs or were given proofs) would anticipate. In essence, it prevents a node from saying "I timestamped Order B at time T1 and Order A at time T2" when in reality A arrived before B. Anyone verifying the VDF chain would detect the inconsistency.

The term "braided" timeline suggests that multiple such VDF chains (from different ingress nodes) could be cross-linked or periodically checkpointed against each other, to avoid any single ingress node diverging in time too far. This is a future enhancement; the initial implementation might rely on each TEE's attested clock plus perhaps periodic synchronization.

2.3 Zero-Knowledge Proofs for Exchange Execution

Using zero-knowledge proofs to validate off-chain computation is a burgeoning area, notably in layer-2 scalability solutions. StarkWare's StarkEx, for example, has been used to verify thousands of trades off-chain and post a proof to Ethereum. zkSync and others similarly can prove batches of transactions or an entire VM execution. Applying ZK proofs to an order matching engine means the engine can be implemented as an off-chain program (with all the speed benefits) but produce a proof (e.g., a SNARK) that some state transition (matching and updating of orderbooks) was done correctly according to the rules.

In Fermi's context, the goal is to prove two key properties about each batch of matches:

1. **Correctness of Matching:** The matching engine followed the defined logic (e.g., it correctly matched available buy and sell orders, didn't create trades that violate price or size constraints, and properly updated the order book state).

2. **Fairness of Ordering:** Given the set of orders and their timestamps, the engine executed them in a way that respects FIFO fairness and price-time priority. This means if two orders were at the same price, the one with the earlier timestamp should be matched first; and no order should be passed over in favor of a later one at the same or worse price.

Proving the first property is similar to what existing exchange rollups do (they ensure the state updates obey certain invariants). Proving the second property may require encoding the condition that the input order list was sorted by timestamp (or that any violation leads to an easily detectable inconsistency). One approach is to have the global sequencer simply sort all incoming orders by timestamp (primary key) and price (secondary key) as part of the off-chain computation, and then the SNARK attests that it output trades by consuming orders in that sorted order. If the sequencer attempted to swap two orders out of order, the proof would not validate against the public input (which might include a Merkle root of all order receipts or timestamps, etc.).

Another challenge for ZK is performance: proving thousands of transactions can be computationally heavy. But recent advancements (and the relatively simple arithmetic of matching) make it feasible. The choice of Groth16 proofs is interesting: Groth16 (used in Zcash, etc.) has very small proof size and fast verification, which is ideal for on-chain verification on Solana (which has limits on transaction size and compute). A downside is needing a trusted setup per circuit, but this

is an acceptable trade-off here given the environment. Alternatively, PLONK or STARKs could be considered; STARKs have no trusted setup and are post-quantum secure, but proofs are larger and verification costs higher (Groth16 on BN254 can be verified in a few milliseconds on-chain, whereas STARK verification might be too heavy for Solana BPF programs without specialized support).

The trust model during the transition is that initially, the global sequencer is run by Fermi Labs (or authorized operators) without a proof. Users rely on reputational trust and the accountability provided by receipts to assume fairness is kept. As the zkVM is integrated, users will gain cryptographic guarantees of fairness without needing to trust the operator, moving the system toward a trustless model akin to a validity rollup. Eventually, multiple parties might run the sequencer program, but only the one that produces the valid proof (and perhaps lowest latency result) is accepted each round.

2.4 Non-Custodial Settlement and Intent Verification

Finally, Fermi's settlement on Solana and the use of signed intents align with the design of several existing DEX protocols that ensure non-custodial execution. The concept of *signing an order intent* means a user digitally signs a message that typically contains: asset to buy, asset to sell, price or price limit, amount, expiration time, and a unique ID or nonce. This signed message is essentially a legally binding (cryptographically) order. The exchange (off-chain) can collect these signed orders and match them, but it cannot alter the terms (e.g., trade a different price or amount) because the signature would then fail to verify on-chain. When a match is found, the exchange submits the two (or more) matched orders to the on-chain contract, which verifies the signatures and the matching conditions (price satisfies both sides, etc.) and then transfers the assets accordingly.

This pattern was established by 0x and later adopted in variants by protocols like AirSwap, dYdX (early version on Ethereum), and others. The key benefit is users maintain custody until the trade, and the trade can only happen exactly as they agreed. In Fermi, because the order matching is more complex (continuous order book, partial fills, multiple matches), the on-chain verification might be a bit more involved: e.g., the contract might need to verify that the trade price is no worse than the limit price in each order, and that the quantities do not exceed the order's open amount. Since the matching engine off-chain will output the specific trade, the simplest model is to treat that off-chain engine as proposing a transaction that uses the two users' pre-approvals (signatures) to move funds.

One crucial design aspect is how funds are managed. Typically, users would deposit funds into the DEX's smart contract (or at least give it permission via an allowance, as in 0x's ERC20 proxy model). On Solana, one common approach is for users to transfer their tokens into a program-controlled vault account associated with their account, or simply have a system of holding tokens in escrow. This ensures that when a match is executed, the tokens are actually there to be swapped. Non-custodial means the program cannot do anything with those tokens unless the user-signed order allows it. Fermi likely adopts a similar approach: users place assets into the Fermi Solana program (or it could use a Serum/OpenBook like model where open orders hold funds). The implementation specifics are beyond our scope, but the high level is that **the protocol itself never unilaterally moves user funds**; every movement is authorized by a user signature corresponding to an order or withdrawal.

This is critical for security and regulation: even if the off-chain sequencer misbehaved, it cannot steal funds or execute unauthorized trades. At worst, it could fail to match orders (censor trades) or attempt an invalid match that the on-chain verification would reject.

In summary, the background analysis shows that Fermi is positioned at the intersection of several strands of research: it takes inspiration from high-speed traditional exchange architectures

(gateway accumulators and price-time priority sequencing), leverages hardware-based trust (like TEEs in proposals such as Chainlink's PROF), employs cryptographic timing (VDF akin to Solana's PoH) and zkSNARK-based verification (similar to rollup systems), and follows a non-custodial settlement paradigm proven by earlier DEX protocols. The combination, as we will see, yields a system that is opinionated in its pursuit of fairness and speed: it does not rely on batching or random ordering to be "fair," but rather enforces a deterministic FIFO order, believing that a well-designed decentralized system can still offer a Wall-Street-like experience without the usual blockchain front-running issues.

3 System Architecture

Fermi's architecture is composed of several layers and components that work together to deliver a unified order book experience. Figure 1 provides a high-level overview of these components and data flows. We describe each component in detail in this section and how they interact to produce a low-latency, fair exchange.

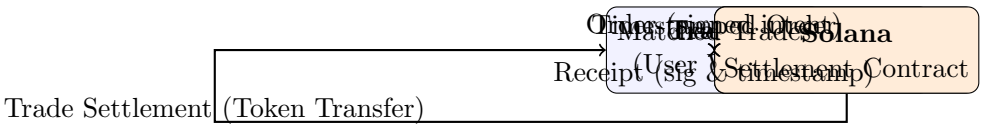

Figure 1: Overview of Fermi Architecture: Traders submit signed orders to their nearest ingress node, which timestamps the order and returns a signed receipt. Local orders are forwarded to the global sequencer, which integrates orders from all ingress nodes and matches them according to price-time priority. Matched trades are sent to the Solana on-chain contract for settlement, which verifies user signatures and transfers tokens accordingly.

3.1 Local Ingress Nodes (Roughtime Timestamps)

Ingress nodes are the first point of contact for users placing orders. One can think of them as analogous to gateway servers or entry points in a traditional exchange (sometimes called Points-of-Presence). In Fermi, ingress nodes serve a crucial role: they provide a low-latency interface for traders while also acting as verifiable timestamping authorities for incoming orders.

Each ingress node leverages third-party *Roughtime* servers to obtain cryptographically verifiable timestamps. By syncing periodically with Roughtime providers, the node can embed a proof of the current time into each order's timestamp. This prevents a malicious operator from arbitrarily forging or backdating timestamps, as any tampering would invalidate the Roughtime proofs.

When a user wants to place an order, they sign the order details with their private key (the same key controlling their funds on Solana). This signed order intent is then sent to an ingress node (likely the one closest to them network-wise, to minimize latency). Upon receiving the order, the ingress node immediately requests a new Roughtime proof (or uses a recently cached proof) and attaches the resulting verifiable timestamp to the order. It then cryptographically signs a receipt containing at least:

- Order identifier (e.g., a hash of the order details and user signature).
- The Roughtime-based timestamp (plus associated proof data).
- The sequencer's signature

- An indicator of acceptance (that the order was received for processing).

This signed receipt is sent back to the user as an acknowledgment. The entire process from the user sending the order to receiving a receipt can be extremely fast (on the order of tens of milliseconds locally, plus network latency). The user is then assured that their order has entered the system at a specific, verifiable time.

Next, the ingress node forwards the order into the core network (to the global sequencer). Under typical network conditions, this happens almost instantly after timestamping. The ingress node may also log the order locally or optionally propagate it to other ingress nodes for redundancy (though in the current design, sending it directly to the global sequencer may suffice as the next stage).

A key point is that ingress nodes do not themselves perform global ordering beyond assigning a Roughtime-verified timestamp. They are like spokes feeding into the hub (the global sequencer). However, because each order carries a cryptographically verifiable timestamp, the global sequencer can rely on those timestamps as a basis for ordering decisions. Essentially, the system pushes the "what time did it arrive?" question to the edges—where network latency from user to ingress is minimal—rather than centralizing timestamp generation.

Security considerations: While removing the TEE reduces certain hardware-trust assumptions, it also places more emphasis on the integrity of the Roughtime protocol and its providers. We mitigate this by:

1. *Planning to add VDF-based verification of timelines* (Section **??**), which further restricts the ability of a malicious node to tamper with timestamps without detection.

2. *Supporting multiple ingress nodes and Roughtime servers.* If a user suspects misbehavior by one node, they can switch to another node or broadcast orders to multiple nodes and compare the timestamps/receipts.

3. *Regular rotation of cryptographic keys* and robust identity mechanisms for ingress nodes, ensuring that any compromised node can be quickly identified and removed from the network.

Overall, by shifting trust from hardware enclaves to independently verifiable cryptographic timestamps, Fermi's ingress design reduces the risk of single points of failure or compromise while preserving low-latency order ingestion and secure receipt generation.

3.2 Client Receipts and Proof of Inclusion/Omission

The signed receipts returned to clients serve as an essential accountability mechanism. In a fair system, if a user submits an order that is valid and should execute given market conditions, that order should either be executed in due time or at least be present in the ordering/matching pipeline. If a malicious sequencer or network tried to "drop" or delay that order unfairly, the user has the receipt as evidence of when and where it entered the system.

How can receipts be used? There are a few scenarios:

- **Proof of Censorship:** Suppose a user's order never seems to execute or even appear in any public feed of the order book, while later orders do. The user can present their receipt to an audit process (or even on-chain if there's a dispute resolution contract) to show that the ingress node acknowledged the order at time T. If the global sequencer did not include it in the sequence of events at around time T, this is evidence of censorship or omission. In a decentralized future, this could trigger a slashing condition for the sequencer (if we have multiple sequencers staking collateral).

- **Proof of Late Execution:** If an order was executed, but at a price-time worse than it should have (e.g., it was queued behind another order that actually arrived later), the user could similarly prove that their order had an earlier timestamp than the other, indicating a fairness violation.

- **Client vs Sequencer Dispute:** If a sequencer claims an order was never received, the client can produce the receipt signed by the ingress node (which the sequencer trusts) to prove it was.

In the current design, because the sequencer is run by essentially the same entity as the ingress (in an initial deployment), disputes would be more relevant when things decentralize. However, already these receipts give users confidence: even if they can't themselves force the sequencer's hand, they know they have evidence in case of wrongdoing, which raises the cost of cheating (reputation or future punishment).

We envision a future on-chain verification mechanism: for example, an aggrieved user could submit their receipt to a Solana contract, perhaps along with some evidence of what the sequencer did (like the sequencer's reported batch of orders around that time). If the contract finds that an order with a valid receipt was unjustly not included, it could slash a bond posted by the sequencer or trigger other recovery logic. This would be akin to a *fraud proof* in an optimistic rollup, except here the fraud is "you left out my transaction or ordered it wrong."

Receipts basically allow the system to implement a form of *censorship resistance*: as long as the ingress node gave a receipt, the sequencer can't easily pretend the order never happened. Of course, one might ask: what if the ingress node itself colludes and doesn't give a receipt or lies? That's where having multiple ingress nodes or eventually a decentralized set of ingress comes in. But within the trust model of one honest enclave per user's submission, the user at least has something signed by the infrastructure.

Additionally, receipts can be used in client-side trading logic. For instance, a market-making bot might submit an order and if it doesn't get a receipt in a timely way, it knows to retry or switch nodes. Once it has a receipt, it can be sure the order is in process and thus start counting time for when it expects either an execution or an expiry.

3.3 Global Sequencer and Matching Engine

The global sequencer in Fermi is akin to the "core engine" of an exchange. It is responsible for:

1. Collecting timestamped orders from all ingress nodes.

2. Determining a total order of all these orders (the sequence in which they will be processed).

3. Executing the matching logic on this sequence, which means updating order books for each trading pair and generating trades when matches occur.

4. Outputting the list of trades (and possibly updated order book state or remaining open orders) to be settled and/or made available to traders as market data.

In the current design, the global sequencer is run as a high-performance server (or cluster) that has connections to all ingress nodes. It likely maintains in-memory order books for each market (trading pair). When a new order comes in, the sequencer determines where it fits:

* It identifies the market (e.g., SOL/USDC). * It uses the timestamp (and perhaps ingress node ID as a secondary ordering to break ties) to place it in a queue relative to other orders that may be concurrently arriving. * It then processes the order: if it is a buy or sell that can trade against

existing orders in the book (e.g., a buy that is higher or equal to the best ask price), it will match it with one or more resting orders. If it fully matches, it may generate one or multiple trades and the order is done (or partially filled). If it is not fully matched, the remaining quantity is placed onto the order book at its limit price, waiting for future matches. * It moves on to the next order in the sequence.

The fairness guarantee we want is that this processing sequence respects the timestamps. Therefore, the sequencer essentially sorts by timestamp. However, to maintain price priority: actually, in an order book, price priority trumps time when comparing two opposite orders. For example, if there is an outstanding sell at $100, and two new buy orders arrive: one is a limit buy at $105 that arrived slightly later, and one is a limit buy at $100 that arrived slightly earlier, the later order will execute first because it crosses the spread (it can match at $100 or up to 105, so it will buy the $100 offer immediately). The earlier buy at $100 will still be queued behind because it cannot match the $100 offer if the $105 order already took it. This is *not* unfair because the $105 order had a more aggressive price.

So price-time priority means:

* Among orders on the same side of the market, time priority is absolute for a given price level. * But an incoming order that crosses can trade immediately regardless of earlier orders that were not willing to trade at that price.

The sequencer's logic naturally does this as long as it processes events in real-time order. Because the act of matching is itself fair if the inputs (orders) are fed in fairly.

One complication: if orders from two different ingress nodes arrive almost simultaneously, how do we order them? If their timestamps are distinct, it's easy (the lower timestamp first). If they have the same timestamp (down to whatever resolution we consider, say milliseconds or microseconds), we need a tie-break. We could break ties by an agreed-upon rule, such as lexicographically by ingress node ID or by a hash of the order (which is unpredictable, effectively random). This tie-break should be consistent and, importantly, if it's cryptographically random (like by hash), it prevents an attacker from deliberately choosing an ingress to advantage their order in ties (since they wouldn't know the hash ordering beforehand ideally). In any event, tie situations are rare and typically have negligible impact.

In the initial implementation, since the global sequencer is just a server taking inputs, it will likely use a simple FIFO queue where messages from ingress nodes are timestamped on arrival at the sequencer as well. Because network jitter could cause slight reordering, it might rely on the ingress-provided timestamp and trust that (which could be a few milliseconds older than arrival). If an order from ingress A with time 1000ms arrives after an order from ingress B with time 1002ms, the sequencer should still place A first due to the timestamp, even though physically it arrived later. This requires the sequencer to buffer and possibly wait a tiny bit to ensure it has received all orders up to a certain time horizon before finalizing order. This is similar to needing to reconcile out-of-order messages. One approach is to use something like Lamport clocks or just assume bounded network delay. If network delay is bounded by, say, 50ms, the sequencer might delay processing an order for 50ms after its stated timestamp to make sure no other earlier timestamped order is still in flight. The specifics can be tuned; a small buffer like that could slightly add latency but ensures fairness.

The global sequencer as implemented will output trades to the Solana contract. It could choose to group trades into batches for efficiency (for example, settle 100 trades in one Solana transaction if possible), or send them one by one. Solana can handle quite a few transactions per second, but grouping might be more efficient if possible. On the other hand, traders want quick confirmation of their trades, so there is a trade-off between batching for throughput and immediate sending for low latency. Fermi might opt for micro-batches (e.g., bundle all trades in a 100ms interval into one

transaction).

The plan to transition the sequencer to a ZK-VM means that the logic described above would be executed inside a virtual machine (potentially a RISC-V or custom arithmetic VM) for which a SNARK proving circuit exists. Each block of operations (say every X seconds or every N orders processed) the VM will output a state hash and a proof that starting from the previous state hash and given the new inputs (orders) it produced the new state correctly. This proof would then be sent to Solana, where a verifier program checks it and then enacts the trades on-chain. In that model, the role of the on-chain component becomes bigger: not only settling trades but also verifying proofs of correctness. Essentially, Fermi would become an *Validium/Volition* style rollup on Solana, with the difference that the data (orders) might not all be on-chain (maybe just the results are).

However, an interim approach might be to run the sequencer off-chain but still post some commitment of the ordering on-chain (like a Merkle root of executed trades or remaining orderbook) along with a proof. This would increase the integrity guarantees even before fully decentralizing who runs the sequencer.

Currently, the global sequencer is trusted for correctness, but the presence of receipts and the eventual ZK upgrade path provide users with increasing assurance. In a production setting, it's likely the Fermi team will run the sequencer and commit to publish all data needed for someone external to reconstruct and verify the sequence (making it a sort of *data availability* guarantee – since if they tried to cheat, external observers with receipts could call them out). This is analogous to how some rollups operate with a centralized sequencer but still cannot easily get away with cheating due to public auditability.

3.4 Solana Settlement Layer

All trades ultimately settle on the Solana blockchain. Solana was chosen for its high throughput and low latency capabilities, which align with Fermi's performance goals. The settlement mechanism is implemented as one or more Solana smart contracts (programs). The main functions of the settlement contract are:

- Maintain custody of user funds (open orders, collateral) in a non-custodial manner.
- Validate and execute trade instructions coming from the sequencer.
- Enforce that trades are derived from authentic user orders by verifying signatures.
- Update balances accordingly (credit the seller with payment, debit the buyer, etc.) and possibly maintain the open order record (like reducing or removing the order that was matched).

When the global sequencer decides on a trade, say User A buys 10 SOL from User B at price X, it will output a message to the Solana program that includes:

* The two orders involved (or references to them), including the original details such as user public keys, order id, price, amount, etc. * The fact that these two orders match for 10 SOL at price X. * Perhaps the signature from each user's order, or a reference to a stored signed order if the contract stores them.

The Solana program will then:

1. Verify that the orders were indeed signed by the users (it can do ed25519 signature checks natively, which Solana supports for its transaction signatures and can also be invoked in programs for any message). The message that was signed likely includes the core order parameters (including a nonce or unique order identifier to prevent replay). 2. Verify that the match is valid given those

orders (e.g., one is a buy order with price ¿= X, the other is a sell order with price ¡= X, and both have enough remaining quantity to fill 10 SOL). 3. If valid, move the tokens: for instance, move 10 SOL from B's escrow to A's escrow (or directly to A's wallet if that's how it's set up) and move payment (10*X in USDC, for example) from A's escrow to B's escrow. Since the program holds or has access to these funds via token accounts, it can do the transfer. 4. Mark the orders as filled by 10 (reduce their remaining amount or mark complete if fully filled). If orders are partially filled, they remain on the book with updated size. 5. Possibly emit an event or log for off-chain monitoring (Solana has logs in transactions).

One important design element: it's inefficient to verify a signature on-chain for each trade if the same order gets matched multiple times (partial fills). A solution is to verify the signature once when the order is first introduced and then store the order in the contract's state with a flag "signature verified, order open with X remaining". Then each trade referencing that order need not provide the signature again, just refer to the stored order data. This is how Serum did it: users place an order via a transaction that stores a new Order in an on-chain order book, and matching happens on-chain. But in Fermi, since matching is off-chain, we might simulate that by having an instruction to "register order" and then later instructions to "match orders". But we want to avoid on-chain registration of every order for performance reasons.

Instead, Fermi might take a hybrid: small orders that get filled quickly never need to be stored on-chain as separate state; their settlement can be done in one transaction that both verifies signature and transfers funds. For larger or long-lived orders, perhaps the sequencer could choose to post them on-chain if they are going to persist (like convert them into a Serum-style order on-chain). However, that reintroduces on-chain load.

Alternatively, we rely on the high capacity: 100k orders per second is off-chain, but 10k trades per second on-chain might be hitting the upper bound of Solana. If every trade is one transaction, 10k TPS is heavy but within plausible range for Solana (Solana has peaks above that). Still, to be safe and to reduce cost, it's better if each on-chain transaction can carry multiple trade settlements. Solana transactions can include multiple instructions and affect multiple accounts, up to some limits. It might be possible to settle, say, 10 or 20 trades in one transaction if they involve different users (so parallelizable).

Another factor is Solana's account model: each token account is owned by a user (or program). For a program to move funds on behalf of user, typically the user must have delegated approval or the program owns the account. Usually, DEXs on Solana create "Open Orders" accounts for each user-market pair that hold their deposits (like Serum did). Possibly Fermi will reuse something like OpenBook's concept or have a similar arrangement where users deposit into vault accounts.

We won't dive too deep, but we ensure to state: **the protocol ensures user funds are never under the control of the off-chain components; only the on-chain program (governed by transparent code) can move funds, and only in ways the user allowed via their signatures.** Therefore, even if the sequencer tried to mismatch trades, the signature check would fail and no unauthorized trade would execute.

Finally, using Solana also gives fast finality (confirmation in ¡ 1 second typically), so users can quickly know their trade is done. It also means the liquidity and assets are directly on a widely-used L1, making integration with wallets and other protocols easier (they can see their balances updated on Solana as soon as settlement occurs).

One might ask: why not settle on a separate L2 or rollup? The choice of Solana is mainly performance and the desire to integrate with Solana's ecosystem. Solana's throughput is high enough to handle settlement, and by taking the order matching off-chain, we avoid burdening Solana with every order message, only the outcomes.

In summary, the settlement pipeline is:

* Off-chain sequencer produces a list of trades (with references to orders). * One or more transactions are formed and submitted to Solana containing these trades. * Solana validators process them: the Fermi program is invoked, it verifies signatures and conditions, and transfers tokens. * The result is that token accounts of users reflect the trades (one's balance goes up, the other down, etc.), completing the exchange.

This design follows the philosophy of "intents": Users sign intents (orders) which are like offers, and the system matches and turns them into final actions (swaps) that are executed on-chain in a trustless way.

4 Sequencing Logic and Fairness Model

A core contribution of Fermi is its sequencing logic that guarantees fairness while maintaining high performance. In this section, we provide a more rigorous description of how orders are processed in a fair manner. We will define what we mean by *FIFO fairness* and *price-time priority* formally in the context of Fermi, and show how the system's design enforces these properties.

4.1 Defining Fairness: FIFO and Price-Time Priority

At a high level, Fermi adheres to the same fairness principles that traditional regulated exchanges do:

- **Price Priority:** An order at a better price has priority over orders at worse prices. For example, a higher bid (buy price) will be matched before a lower bid, regardless of time, because it's more competitive. This ensures economic efficiency (best prices get executed first).
- **Time Priority (FIFO) within the same price:** Given two orders of the same type (e.g., both are buy orders) and at the same price, the one that arrived (was timestamped) earlier will have priority in execution over the one that arrived later. This is the "first-in, first-out" principle. If Order A and Order B are both selling at $100, and A arrived before B, then A's quantity will be exhausted (through matches with buyers) before B gets to sell anything at $100.

In the context of an order book, these rules translate into how the matching engine operates:

* The order book for each side (buy or sell) is typically sorted by price and then by time of arrival. * When a new order comes in, it will match against the opposite side starting from the best price and going down/up, and within a given price level it will match against the oldest order first.

Now, how do unfair outcomes arise, and what must we prevent?

* **Late-comer jumping queue:** If an order that arrived later gets executed before an earlier order at the same price, that's a fairness violation (front-running). This could happen if the sequencer maliciously reorders or if there's some system-induced distortion of time. * **Order omission:** If an order is dropped and never executed even though it was valid and should have been (e.g., there were matching opportunities), that is unfair to the user. * **Price inversion:** If a worse price order somehow executes while a better price order was available and not executed, that's a violation (though usually this would also reflect a time issue or a bug). * **Frontrunning by insiders:** If the sequencer or an ingress node operator sees a large order and sneaks in an order of their own before processing it (despite it arriving later), that is precisely the kind of scenario we aim to prevent.

Fermi's fairness model essentially says: the system behaves as if all orders were processed exactly in the chronological order they were submitted by users (excluding network propagation delays). More formally, if two orders o_1 and o_2 are submitted and $o_1 s submission(to a ingress node) timestamp is less than o_2 s$ will be fully processed (matched or posted) before o_2 is processed, or if o_2 affects the market first (e.g. o_2 trades with some orders), then it must be the case that o_1 could not have affected those trades (for instance, o_1 was on the same side or at a worse price so it wasn't in direct competition). In other words, no ordering inversion will cause a user to lose a trade they should have won under FIFO.

This is enforced by the global sequencer's algorithm, which we will describe stepwise:

1. The sequencer maintains a priority queue of incoming orders. The priority key is primarily the timestamp, secondarily something like ingress ID or a random tie-break to ensure a deterministic order if equal timestamps occur.

2. It continuously takes the next order from this queue (the one with the smallest timestamp that hasn't been processed yet) and processes it against the current order book state.

3. Because the queue is sorted by timestamp, this inherently ensures FIFO processing order. The only nuance is as discussed: a later timestamp order with a better price can "jump ahead" in terms of outcome, but not in terms of processing. It will simply cause the earlier order (at worse price) to remain unfilled. That's fine, because the earlier order didn't want to pay that higher price or sell at that lower price.

4. The matching engine is memory of all resting orders and will update as it goes. By the time it gets to the later order, earlier ones either are gone or waiting.

One might worry about the granularity of timestamps. If we stamp at, say, microsecond resolution, what if two orders arrive in the same microsecond? We have the tie-break rule for that (could be a deterministic but essentially arbitrary choice which we consider fair since, from user perspective, they were virtually simultaneous and no one had knowledge to exploit such tiny differences).

4.2 Latency and Race Conditions

In a distributed system, there is always a window where two traders might be racing. For example, two traders see a price and both try to send an order to take it. Who gets it? In a centralized exchange, it's the one who arrives first at the matching engine. In Fermi, it's the one whose order gets timestamped first at an ingress node and then delivered to sequencer. Because we place ingress nodes close to users (potentially one in each major region), we reduce the network latency for the initial hop. However, someone physically closer to an ingress node might consistently get slightly earlier timestamps than someone farther away, which is a form of geographic advantage. This is similar to how in traditional markets co-location near an exchange gives an advantage. While we cannot remove the physics of latency, Fermi's design tries to make it as fair as possible by having multiple ingress points. If all traders connect to their nearest ingress, then the latency to the system is roughly equal to latency to the nearest data center. The race then is decided by those few milliseconds differences and system processing times, which is akin to existing electronic markets. The difference with blockchain normally is that you have unpredictability in when a block is mined, etc., which Fermi avoids by having continuous processing.

Another aspect is fairness across ingress nodes: what if one ingress node delays forwarding orders? If an ingress node is maliciously delaying forwarding to help others, the VDF and receipts architecture will catch it. All ingress could be required to forward orders to sequencer immediately

and potentially share them with each other, so that even if one ingress fails, another can supply the order to sequencer (using the receipt as proof maybe). But in current design, we trust ingress to forward properly (especially if run by same operator). In decentralization, to ensure fairness, ingress nodes might gossip orders among themselves, or there could even be a second stage where ingress nodes form a mini-consensus on the order of orders in a given epoch (but that might reduce performance).

We plan to implement monitoring such that if an ingress node's orders consistently show up delayed at the sequencer relative to others (beyond expected network difference), that node can be flagged.

4.3 Groth16 Proofs of Fair Ordering

Once the sequencer's logic is moved into a zk-prover, how do we prove fairness? In the proof circuit, we will likely include the following:

* The input will include a list of orders with their timestamps and details. * The program sorts this list by timestamp (and tie-break rule). * Then it simulates the matching. * It outputs the trades. * We enforce in the circuit that the list was indeed sorted by timestamp (meaning no timestamp later comes before an earlier one). * We also enforce matching logic.

If the sequencer tried to feed the prover a list that is out-of-order (like swapped two orders), the sorting check would fail and the proof wouldn't generate (or would be invalid). If they omit an order, that order wouldn't be in the list, but if we also give the circuit as input a Merkle root of all receipts that should be included (posted by ingress nodes on chain maybe), then omission could be detected. More practically, proofs might initially focus on matching correctness, and fairness proofs might come from an external mechanism like fraud proofs with receipts as described.

However, a SNARK could indeed cover fairness if it had a complete view of all orders. It might be heavy to include every single order's data as a public input to an on-chain proof verification. Instead, one could:

* Have each batch of orders anchored by a Merkle root (of say all order data in that batch). * The SNARK proves "I took the last known order book state, and a set of new orders (whose Merkle root is X, provided as a public input), processed them fairly, and the resulting state is Y and trades list has hash Z." * Then on chain, one might not verify every single order, but people off-chain could check if their order (with a receipt) was included in that Merkle tree X. If not, they can challenge out of band or refuse to accept the state.

This becomes somewhat like an optimistic rollup with validity proofs for state but data availability outside. Achieving data availability is an issue – perhaps the receipts themselves are the data availability. If all users have receipts, they know their orders. If the sequencer tries to exclude orders, the user will notice their order not executed and can present receipt as we discussed.

Anyway, the end goal is that users and observers have cryptographic assurance the matching was fair. Either via direct validity proof (strongest) or via a combination of validity proof + censorship challenge mechanism.

One intermediate step could be: run the sequencer off-chain but every block or interval, post a *commitment* to Solana of all new order receipts (like a root of receipts). This way, even without full ZK, we are committing to the set of orders seen. If later, an order with a receipt isn't reflected in trades or the order book, one can point to the commitment that it was supposed to be there. This is like making order data available on chain (in compressed form) which helps watchers.

We should also mention the trust model: Initially, Fermi's fairness is ensured by the operator's incentives and the auditability with receipts. Later, it's ensured by cryptography. During the

transition, the system might still be semi-permissioned (maybe one sequencer), but with proofs such that even that sequencer cannot cheat without the proof failing.

4.4 Economic Security and Incentives

Fairness doesn't only come from technology; it also comes from aligning incentives such that cheating doesn't pay. In a future state where multiple sequencers could compete or take turns (like block proposers), each would likely have to put down a stake or bond. If one is caught cheating (through a receipt proof or a failed zk proof), that stake can be slashed. This economic deterrent will discourage any rational actor from attempting to deviate, because the expected loss (losing stake, losing reputation, being kicked out) outweighs any potential short-term gain (which in a system with proofs might be zero anyway, since you can't even successfully cheat without getting caught).

One might consider the role of fees: The sequencer could earn fees per trade or order (like how rollup operators earn fees). As long as those fees are not heavily dependent on reordering, the sequencer is economically motivated to include everyone's orders (more orders, more fees) rather than exclude someone. Censorship might only occur if including someone's order harms the sequencer's own trading interests. To mitigate that, maybe sequencers (especially if decentralized) shouldn't be active traders themselves or their trades could be flagged by design. Or one could introduce an auction for the right to sequence (like Flashbots but fairness oriented) – but that gets complex and might reintroduce MEV as something sold (which is contrary to fairness for users).

Instead, Fermi likely will keep sequencers as infrastructure providers who just earn fees, and if it decentralizes, maybe it elects them via stake and performance, not via payment for ordering.

In summary, the fairness model of Fermi is that of a traditional exchange – price-time priority – enforced by distributed timestamping and verifiable processing. It brings blockchain elements (like proofs and slashing) to ensure that this exchange-like behavior can be trusted without a central authority, even in the face of adversarial conditions.

5 Light Protocol Compressed Tokens for Enhanced Scalability

A critical aspect of sustaining high throughput on Solana while managing the potentially large state overhead of active orders and partially filled positions is the adoption of light protocol compressed tokens. Recent developments in Solana's on-chain data compression primitives (e.g., state compression for fungible and non-fungible tokens) have opened the door to significantly more efficient storage and transfer of token metadata at scale[14]. Leveraging these techniques in Fermi can greatly reduce on-chain resource consumption, thereby improving both cost efficiency and throughput.

5.1 Motivation for Compression on Solana

Despite Solana's relatively high transaction capacity (often cited in the range of thousands of TPS in production environments), any protocol managing large numbers of open orders or partial fills must be mindful of the associated state footprint. Each new token account, each incremental balance entry, and each side of an open order typically incurs memory and compute overhead. While the off-chain matching architecture used by Fermi already alleviates much of the per-order storage cost, the settlement layer still must deal with on-chain accounts representing asset ownership, escrowed funds, or partially filled positions.

Solana's compression model[15] allows a program to store token data in a Merkle tree that resides largely off-chain (e.g., in an Arweave-like storage or in a specialized Solana compression structure) while anchoring only the root of this tree on-chain. Data proofs can be verified against the on-chain root, enabling trust-minimized verification of the underlying tokens. This approach drastically reduces the bytes stored directly in Solana accounts, which in turn reduces costs and resource contention.

Fermi can take advantage of these compressed tokens in two ways:

1. **Ephemeral Balance Tracking:** Instead of maintaining a dedicated token account for every partially filled or resting order, we can issue a "compressed balance token" representing the user's claim on a portion of liquidity. The program verifies the integrity of this claim on-chain via Merkle inclusion proofs. This reduces the creation of multiple dedicated token accounts, freeing up Solana account space and lowering rent or storage fees.

2. **Aggregate Position Representation:** When a user's liquidity is split across multiple markets, instead of scattering the user's escrowed balances into distinct accounts across many order books, we can maintain a single "compressed position token" that references the user's entire deposit. Off-chain, Fermi's sequencer and order book logic track the partial allocations. On-chain, the user's compressed token claims a global amount of assets that the user can settle in partial increments as orders match. This structure offloads the fine-grained distribution of liquidity into an off-chain proof system (aligned with Fermi's general off-chain matching and on-chain settlement approach).

The net effect is improved scalability: smaller on-chain footprints per user and per order, lower fees, and higher effective throughput for real trading activity. By integrating compressed tokens, Fermi ensures that the bottleneck of on-chain state is pushed further away, allowing the protocol to handle tens of thousands of trades per second without bloating the ledger.

5.2 Technical Considerations and Implementation

1. **Merkle Tree Construction:** In a typical Solana compression model, one constructs a Merkle tree where each leaf represents a token ownership record (or partial balance). Fermi's architecture can group user balances or user "order-lot" claims (e.g., 100 USDC allocated to a specific order) into leaves. The root of this tree is stored on-chain in a "compression program" account. When a user places or cancels orders, the sequencer updates the Merkle structure off-chain and eventually posts a new root on-chain after settlement batches.

2. **Inclusion Proofs:** To redeem or settle any portion of a compressed token, the user (or the Fermi sequencer on behalf of the user) provides a Merkle inclusion proof showing that the leaf representing the user's allocation is indeed part of the compressed tree. The Solana program verifies the proof against the on-chain root. If it succeeds, the program knows the user's claim is legitimate and can release the appropriate underlying tokens.

3. **Concurrent Updates and Batching:** Because many trades may settle in a short time frame, the protocol typically batches updates to the compressed root. The global sequencer or a specialized aggregator can compute these batched updates in real-time, produce an updated Merkle root, generate small proofs for each user that must settle immediately, and finalize it on-chain. This design parallels "rollup" data-availability approaches, but specialized to Solana's compression primitives.

4. **Security Model and Trust:** The correctness of compressed tokens depends on robust proofs that the Merkle root on-chain is consistent with the off-chain data. Just as with the rest of Fermi's approach—where off-chain matching is provably correct—the system can incorporate zero-knowledge or other cryptographic proofs to assure that the transitions in the compressed tree (e.g., the partial fill of an order) are valid. If there is any discrepancy, a user can challenge or present receipts. As the compressed token roots are posted to Solana, a malicious operator cannot unilaterally alter or omit user balances without detection.

5. **Integration with TEEs and VDFs:** Since each local ingress node and the global sequencer produce receipts and maintain a verifiable timeline (via TEEs and the planned VDF-braided approach), the updates to the compressed token tree remain traceable. Each order fill or partial fill is tied to a specific timeline event; the update to the user's compressed balance must reflect that event in the Merkle path. This synergy ensures that the final state of user balances on-chain is consistent with the fair ordering rules.

Several high-throughput projects on Solana have demonstrated the feasibility of large-scale compressed state[16]. By aligning Fermi's data model with these compression techniques, the protocol can maintain the non-custodial and verifiable nature of on-chain settlement while multiplying the number of trades it can handle at any given time. As compression technology matures, we expect further optimizations—such as more efficient Merkle tree updates and potential synergy with new cryptographic primitives (e.g., STARK-based proofs)—to help Fermi scale to an even greater magnitude of daily trading volume.

6 Virtual Liquidity and Multi-Order Collateralization

Another powerful technique to maximize capital efficiency in an orderbook-based DEX is virtual liquidity, wherein the same pool of collateral can simultaneously back multiple open orders under a unified risk framework. Traditional centralized prime brokerages and margin systems have long employed such techniques to allow traders to leverage a single collateral balance for multiple concurrent positions, provided that their net exposures are within safe limits[17]. Introducing virtual liquidity into Fermi's design can significantly amplify the effective liquidity on the order book, enhancing market depth and reducing spreads.

6.1 Concept of Virtual Liquidity

In a typical DEX orderbook, each order is fully collateralized independently. For instance, if a user places a buy order for 100 SOL at $20, they must lock up $2,000 (in USDC) in an escrow account. If the same user also places a separate buy order for 50 SOL at $18, they lock up an additional $900, and so on. This siloed approach ensures solvency but is capital-inefficient, as the user's maximum possible fill across all orders is usually less than the sum of those notional amounts. In practice, most orders will not all fill at once unless markets move significantly.

Virtual liquidity (sometimes called cross-margining or multi-order collateralization) instead treats the user's total deposited balance as a shared resource. If the user has $3,000 in USDC collateral, they can post several buy orders whose combined notional might exceed $3,000 in the strict sum, as long as the user's net exposure at any potential fill scenario remains within the safe collateral limit. Concretely, if their orders are at different price levels, or if only some subset can be filled before the user's available balance is consumed, the protocol can allow them to place multiple "virtual" orders. These orders remain valid but share the same underlying collateral. As soon

as one is filled, the user's effective available balance decreases, and the others may be partially invalidated or reduced to prevent over-extension.

6.2 Implementation in Fermi

In Fermi, implementing virtual liquidity requires careful off-chain logic in the matching engine as well as robust on-chain settlement checks:

1. **Unified Collateral Account:** Instead of creating a separate escrow for each order, a user deposits funds (e.g., USDC) into a single "unified margin account" under the Fermi settlement program. This margin account is tracked by the program, possibly using compressed tokens (Section 7) to represent the total deposit. Off-chain, the user can place multiple orders referencing this same margin account.

2. **Off-Chain Risk Checks:** The global sequencer (or eventually the zk-VM inside which the matcher runs) must verify that any new order does not push the user's potential exposure beyond their margin capacity. This check can be done by simulating the worst-case fill scenario. For example, if two orders could fill simultaneously under certain market conditions, the system sums up both exposures to ensure they do not exceed the user's collateral. If that sum would surpass the user's margin, the second order must be partially restricted (or disallowed) to keep total risk in line.

3. **On-Chain Settlement Logic:** Because these checks occur off-chain, it is essential that the on-chain contract re-validates solvency whenever an order is about to be settled. When a matched trade is submitted for settlement, the Solana program confirms that (a) the user's margin account still holds enough unencumbered collateral to cover the cost, and (b) the order's signature and size are valid. If valid, the trade proceeds; otherwise, it fails, effectively voiding the overextended portion of the order. By requiring each settlement to pass the margin test, the protocol eliminates the risk of "double-spend" across multiple orders.

4. **Partial Fill Adjustments:** Once one of the user's orders is filled (entirely or partially), the user's margin usage changes. The off-chain engine recalculates how much collateral is left available to support other open orders at their respective limit prices. If needed, the sequencer adjusts (down-sizes) or cancels any order that would now exceed the user's updated margin capacity. This entire adjustment can later be proven correct via receipts (showing original orders) and by zero-knowledge proofs (verifying that no order was wrongly included or excluded).

5. **Complex Exposures and Derivatives:** Virtual liquidity can extend beyond spot trades. If Fermi supports margin trading or derivatives, the same principle applies: a user might have cross-margin for multiple instruments—spot SOL, a SOL/USD perpetual, an option position, etc.—all backed by a single deposit. The net risk is evaluated off-chain by the matching engine's risk module, and the on-chain settlement checks final collateral sufficiency at the moment of actual transfer. This multi-instrument approach is an active area of research and has been explored by protocols like dYdX (for cross-margin) and certain prime-broker DeFi solutions[18].

6.3 Benefits and Challenges

- **Enhanced Liquidity:** By removing the requirement that each order be fully funded independently, traders can post more orders at diverse price points. This leads to tighter spreads

and deeper books, as the same capital can effectively "cover" multiple possible trades.

- **Capital Efficiency:** Market makers and algorithmic traders benefit significantly from cross-margin setups, since they no longer have to fragment their capital across dozens of price levels. This efficiency often leads to higher volumes, which in turn benefits the entire ecosystem via lower slippage and better price discovery.

- **Risk Management Complexity:** Allowing a user to overcommit notional value relative to their deposit requires a robust real-time risk calculation. The worst-case fill scenario can be complex if multiple orders are on correlated markets. This is especially true for advanced instruments (perpetuals, options) where partial fills might trigger changes in margin requirements. Fermi's design addresses this by (1) deferring ultimate settlement checks to the on-chain program, and (2) using the global sequencer's high-performance computing environment (and eventually zk-proofs) for correct, transparent off-chain risk modeling.

- **Front-Running and Liquidations:** If a user's margin is on the brink of being exhausted, an unexpected large fill could prompt an immediate liquidation event. While Fermi's fair ordering mechanism (Section 4) largely prevents front-running of user trades, the protocol must still handle the standard DeFi challenge of ensuring timely liquidations. This can be integrated into the settlement layer: if a fill drives the user's margin into deficit, the settlement contract or an auxiliary liquidation program halts further trades and triggers a partial liquidation of positions.

6.4 Related Work and Citations

Virtual or cross-margin liquidity is a well-studied concept in both traditional finance and crypto:

- **Traditional Central Counterparties (CCPs):** Large clearinghouses (e.g., CME Clearing, LCH) allow offsets between positions in correlated contracts, effectively reducing margin requirements[19].

- **DeFi Implementations:** dYdX's cross-margin feature on Ethereum rollups[18], and derivatives protocols like Injective and GMX, have introduced multi-market collateralization, albeit at smaller scale due to L1 constraints.

- **Prime Brokerage Models:** In CeFi, prime brokers allow multiple instruments to share the same collateral. Virtual liquidity in Fermi is akin to a decentralized prime broker solution, with cryptographic receipts replacing the bilateral trust.

By incorporating virtual liquidity, Fermi will empower traders to deploy their capital more flexibly without compromising solvency. The synergy with compressed tokens (Section 7) further optimizes on-chain resource usage: a single compressed margin position can be subdivided among many potential trades off-chain, and only settle the final executed trades on-chain.

7 Conclusion

We have presented Fermi, a next-generation decentralized exchange protocol that seeks to marry the speed and user experience of traditional order book exchanges with the transparency and trustlessness of blockchain. Fermi's design is characterized by a relentless focus on three pillars: **performance**, **fairness**, and **verifiability**. Through a combination of architectural choices —

including local TEE-based order ingestion, cryptographic timestamping via VDFs, a provably fair global sequencing mechanism, and on-chain settlement with user-signature enforcement — Fermi ensures that trades occur in a fast yet fair manner. Traders no longer need to worry about hidden intermediaries reordering transactions for profit; every step from order placement to execution either is trustlessly automated or can be audited and verified.

Our exploration of the system internals shows that it is feasible to handle on the order of 10^5 TPS off-chain and about 10^4 TPS on-chain, which would make Fermi one of the highest throughput DEXs in existence. More importantly, it achieves this without resorting to central custody or opaque matching: users keep control of their funds until the very moment of trade, and even then, funds move under the strict conditions of a smart contract that enforces the users' intents. The use of Solana as a settlement layer provides the high-performance backbone and global state machine to anchor Fermi's activity, leveraging Solana's strengths in throughput and quick finality.

We also outlined how Fermi will progressively decentralize and harden its security. In future iterations, no single entity will need to be trusted for correct operation: the global sequencer will produce zkSNARK proofs attesting to the correctness and FIFO fairness of every batch of matches, and a network of staked nodes will collaboratively run the system with incentives aligned to follow the protocol rules. Fraud proofs and slashing will act as a strong deterrent against any deviation, making it economically irrational to attempt to cheat. In essence, the eventual goal is that Fermi operates as a public utility-like network, where many participants can contribute to its operation, and the system's correctness emerges from cryptographic guarantees and game-theoretic equilibrium rather than from reliance on any single party's honesty.

The path forward involves significant engineering and research: implementing efficient proving circuits for the matching engine, optimizing VDF integration, and designing a sound mechanism for decentralizing the sequencer role without sacrificing too much latency. These are areas of active development. Moreover, extensive testing and audits will be needed, given the complexity of the system (combining novel hardware, software, and cryptographic components). We intentionally leave out considerations of tokenomics or fee models in this paper, as our focus is on the technical architecture; however, those will be detailed in a separate work, as they are important for bootstrapping and sustaining the network.

Fermi represents a bold step towards a fair financial infrastructure. By enforcing price-time priority and preventing front-running at a protocol level, it offers a solution to the chronic issue of MEV in DeFi, giving users an exchange where the rules mimic the best practices of traditional markets but without their centralization drawbacks. With performance that rivals centralized systems and the inherent advantages of DeFi (self-custody, global accessibility, transparency), Fermi can be the foundation for a new wave of high-frequency, low-latency trading applications on blockchain. We envision a future where any trader, from a small retail user to a large institution, can interact on a level playing field in the Fermi ecosystem, confident that the fastest wins are earned by network speed and not by exploitative ordering games, and that the system's fairness is underwritten by code and math rather than the goodwill of an intermediary.

References

References

[1] Philip Daian, Steven Goldfeder, Tyler Kell, Yunqi Li, Dawn Song. Flash Boys 2.0: Frontrunning, Transaction Reordering, and Consensus Instability in Decentralized Exchanges. *arXiv:1904.05234*, 2019. https://arxiv.org/abs/1904.05234

[2] Mahimna Kelkar, Fan Zhang, Steven Goldfeder, Ari Juels. Order-Fairness for Byzantine Consensus. *IACR Cryptology ePrint Archive 2020/269*, 2020. https://eprint.iacr.org/2020/269.pdf

[3] Mahimna Kelkar, Soubhik Deb, Sishan Long, Ari Juels, Sreeram Kannan. Themis: Fast, Strong Order-Fairness in Byzantine Consensus. In *CCS '23*, pp. 475–489, 2023. https://dl.acm.org/doi/pdf/10.1145/3576915.3616658

[4] Andrei Constantinides, Diana Ghinea, Lioba Heimbach, Zilin Wang, Roger Wattenhofer. A Fair and Resilient Decentralized Clock Network for Transaction Ordering. *OPODIS 2023*, Article No. 8, 2023. https://arxiv.org/abs/2305.05206

[5] Eric Budish, Peter Cramton, John Shim. The High-Frequency Trading Arms Race: Frequent Batch Auctions as a Market Design Response. *Quarterly Journal of Economics*, 130(4):1547–1621, 2015. https://academic.oup.com/qje/article/130/4/1547/1916146

[6] Ari Juels, Lorenz Breidenbach, Florian Tramèr. Fair Sequencing Services: Enabling a Provably Fair DeFi Ecosystem. *Chainlink Blog*, September 2020. https://blog.chain.link/chainlink-fair-sequencing-services-enabling-a-provably-fair-defi-ecosystem/

[7] Kushal Babel et al. PROF: Protected Order Flow in a Profit-Seeking World. *IACR Cryptology ePrint Archive 2024/1241*, 2024. https://eprint.iacr.org/2024/1241

[8] Will Warren, Amir Bandeali. 0x: An Open Protocol for Decentralized Exchange on the Ethereum Blockchain. Whitepaper, February 2017. https://github.com/0xproject/whitepaper/blob/master/0x_white_paper.pdf

[9] Project Serum. Serum Whitepaper. July 2020. https://assets.website-files.com/61382d4555f82a75dc677b6f/61384a6d5c937269dbed185c_serum_white_paper.88d98f84.pdf

[10] Anatoly Yakovenko et al. Solana: A New Architecture for a High Performance Blockchain (v0.8.13). 2018. https://solana.com/solana-whitepaper.pdf

[11] StarkWare Industries. StarkEx Documentation. 2023. https://docs.starkware.co/starkex/index.html

[12] Matter Labs. Introducing zkSync: The Missing Link to Mass Adoption of Ethereum. Blog post, October 2019. https://blog.matter-labs.io/introducing-zk-sync-the-missing-link-to-mass-adoption-of-ethereum-14c9cea83f58

[13] Jens Groth. On the Size of Pairing-Based Non-interactive Arguments. In *Eurocrypt 2016*. https://eprint.iacr.org/2016/260.pdf

[14] Metaplex Foundation, "Compressed NFTs: Scaling NFT Ownership on Solana," 2023.

[15] Solana Labs, "State Compression Implementation Documentation," 2024.

[16] Helium Network, "HIP 70: Migrating Helium to Solana Using Compressed NFTs," 2023.

[17] Duffie, D., "Futures Markets," Prentice Hall, 1989; also see "prime brokerage" margin frameworks in T. Copeland & J. Weston, Financial Theory and Corporate Policy.

[18] dYdX Trading Inc., "Cross-Margin System for Perpetual Contracts," 2022.

[19] CME Clearing, "Portfolio Margining and Offsets," 2021.